TRIBHUVAN UNIVERSITY

PATAN MULTIPLE CAMPUS

PATANDHOKA, LALITPUR

LAB REPORT

SOFTWARE ENGINEERING (BIT 302)

SUBMITTED BY:

NAME: SURESH DAHAL
CLASS: BIT – III/I
ROLL: 23
DATE: 2082-12-05

SUBMITTED TO:

DEO NARAYAN YADAV

CHECKED BY

LAB-3: GIT AND GITHUB

THEORY:

Git is a distributed version control system used to track changes in source code during software development. It allows multiple developers to work on a project simultaneously, maintain different versions of code, and manage changes efficiently. Git stores the complete history of a project, enabling users to revert to previous versions when needed and ensuring data integrity.

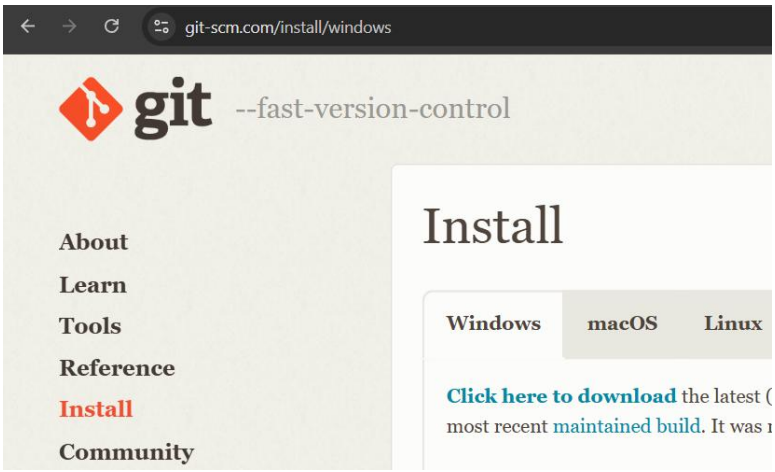
GitHub is a web-based platform that uses Git for version control and provides online repository hosting. It enables developers to store, share, and collaborate on projects from anywhere. GitHub also offers additional features like pull requests, issue tracking, and project management tools, making teamwork and code management more organized and efficient.

PROCEDURE:

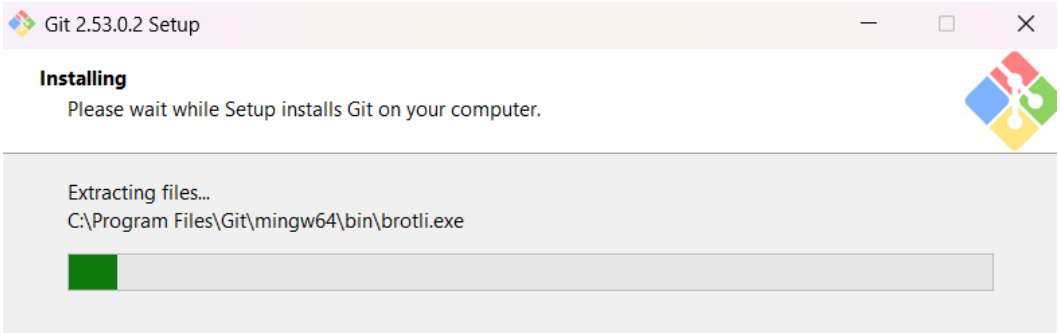
Below is the procedure for installing and using Git and GitHub:

INSTALLATION OF GIT:

- Go to https://git-scm.com/install and download installer



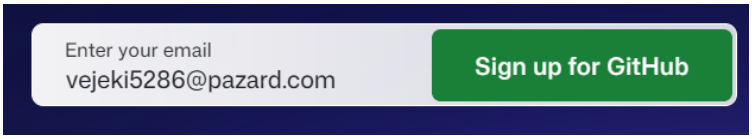
- Run the installer



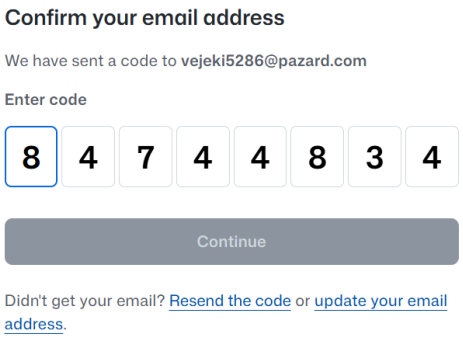
Now the Git is installed on the computer.

CREATION OF GITHUB ACCOUNT:

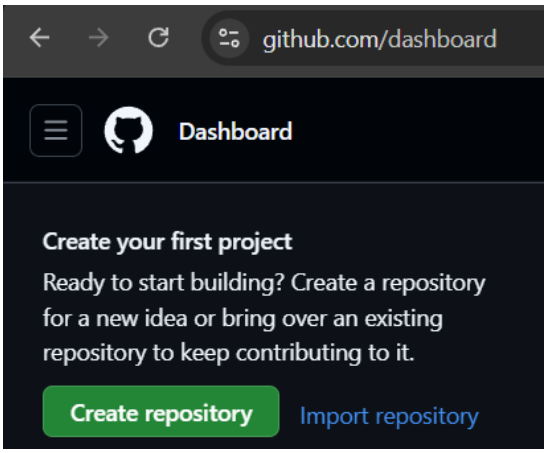
- Go to https://github.com , enter email address and click on sign up



- Fill the necessary information and click on register
- Verify email address



- Login to GitHub account



CREATING NEW REPOSITORY IN GITHUB:

- We can create a new GitHub repository by clicking on the Create Repository button on GitHub dashboard
- We can also create new repository by clicking on the profile icon > repositories > new

CLONING THE REPOSITORY:

To clone the GitHub repositories, we use git clone command

- Open git bash
- Type git clone <repository_url> and hit enter

```
MINGW64:/c/Users/VICTUS

VICTUS@Aconitin MINGW64 ~
$ git clone https://github.com/patanStudent/swe-practical
Cloning into 'swe-practical'...
```

GIT BRANCHES:

A branch in Git is a separate line of development that allows developers to work on new features, bug fixes, or experiments without affecting the main codebase. It helps in maintaining stability while enabling parallel development. Changes made in a branch can later be merged into the main branch.

Git has default branch called main or master. We can use git checkout command to create a new branch.

- Open cloned repository folder in Git bash
- Use git branch command to view branches in that repository

```
VICTUS@Aconitin MINGW64 ~/Desktop/suresh/
$ git branch
* main
```

- Create new branch using git checkout command

```
VICTUS@Aconitin MINGW64 ~/Desktop/suresh/
$ git checkout -b new-feature
Switched to a new branch 'new-feature'

VICTUS@Aconitin MINGW64 ~/Desktop/suresh/
$ git branch
  main
* new-feature
```

- Use git checkout command to switch the branch

```
VICTUS@Aconitin MINGW64 ~/Desktop/suresh/
$ git checkout main
Switched to branch 'main'
Your branch is up to date with 'origin/ma

VICTUS@Aconitin MINGW64 ~/Desktop/suresh/
$ git branch
* main
  new-feature
```

The key difference is that we use -b while creating a new branch but not while switching to the existing branch.

CHECKING GIT STATUS:

Checking status of a project is straightforward, we use git status command to see tracked, untracked, deleted, modified files.

- Use git status command to view status of a git repository

```
VICTUS@Aconitin MINGW64 ~/Desktop/suresh/coderSuresh (main)
$ git status
On branch main
Your branch is up to date with 'origin/main'.

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        new_file.txt

nothing added to commit but untracked files present (use "git add" to track)
```

MAKING COMMIT IN GIT:

A commit in Git is a snapshot of changes made to the project files at a specific point in time. It records modifications along with a message describing the changes, allowing developers to track progress and maintain the history of the project. It helps in reverting to previous versions and ensures better management of code changes.

- Use Git add command to add modified files to staging area of Git
- Use Git commit command to track the changes

```
VICTUS@Aconitin MINGW64 ~/Desktop/suresh/
$ git add new_file.txt
warning: in the working copy of 'new_file
next time Git touches it

VICTUS@Aconitin MINGW64 ~/Desktop/suresh/
$ git commit -m "add new file"
[main 7ed7418] add new file
 1 file changed, 1 insertion(+)
 create mode 100644 new_file.txt
```

PUSHING CHANGES TO GITHUB:

Pushing means to upload local changes to remote GitHub repositories.

- Use git push command to push the changes

```
VICTUS@Aconitin MINGW64 ~/Desktop/suresh/coderSuresh (main)
$ git push origin main
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 12 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 296 bytes | 296.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote: This repository moved. Please use the new location:
remote:   https://github.com/coderSuresh/codersuresh.git
To https://github.com/coderSuresh/coderSuresh
   48a9a96..7ed7418  main -> main
```

STAGING CHANGES IN GIT:

Staging refers to the process of selecting and preparing specific file modifications before committing them to the repository.

- Use Git add command to stage changes

```
VICTUS@Aconitin MINGW64 ~/Desktop/suresh/
$ git add new_file.txt
```

- We can do "git add ." to stage all changes

VIEWING CHANGES WITH GIT DIFF

- Use Git diff command to view difference. It shows changes to existing files.

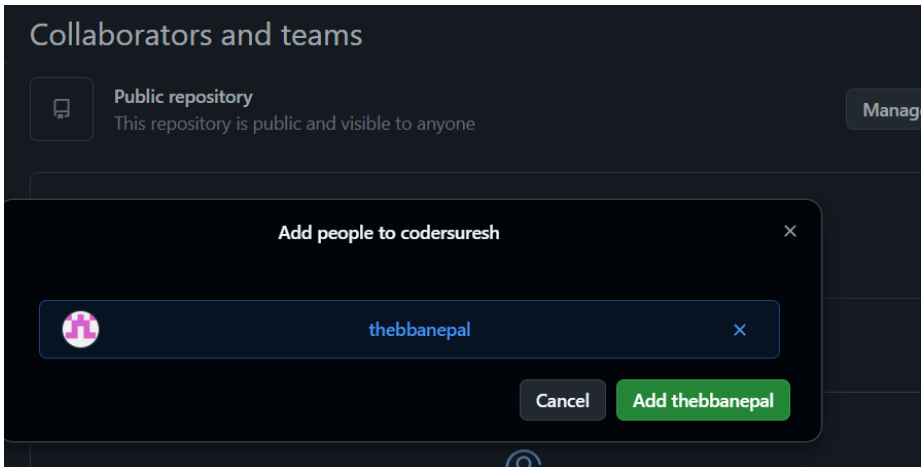
```
VICTUS@Aconitin MINGW64 ~/Desktop/suresh/coderSuresh (main)
$ git diff
warning: in the working copy of 'file.txt', LF will be replaced by CRLF the next
 time Git touches it
diff --git a/file.txt b/file.txt
index aa83006..0c2aa8a 100644
--- a/file.txt
+++ b/file.txt
@@ -1 +1 @@
-random file
+not a random file

VICTUS@Aconitin MINGW64 ~/Desktop/suresh/coderSuresh (main)
```

COLLABORATION IN GITHUB:

GitHub allows multiple developers to work on a single project.

Adding New Collaborator:

- Go to GitHub repository > setting > collaborators
- Search and add new collaborator



Now, added users can see the repository even if it is private and can work on it. There are rules to decide whether to allow them to push directly to the repository or make pull requests compulsory to go through the code review process.

The collaborator gets an invitation and accepts it.

Collaborator Clones the Repository:

Collaborator clones the repository into the local machine using git clone command.

Collaborator Creates a Feature Branch:

Collaborator creates a new branch to work on a specific feature using git checkout command.

```
@thebbanepal ➜/workspaces/codersuresh (main) $ git checkout -b new-feature
Switched to a new branch 'new-feature'
@thebbanepal ➜/workspaces/codersuresh (new-feature) $
```

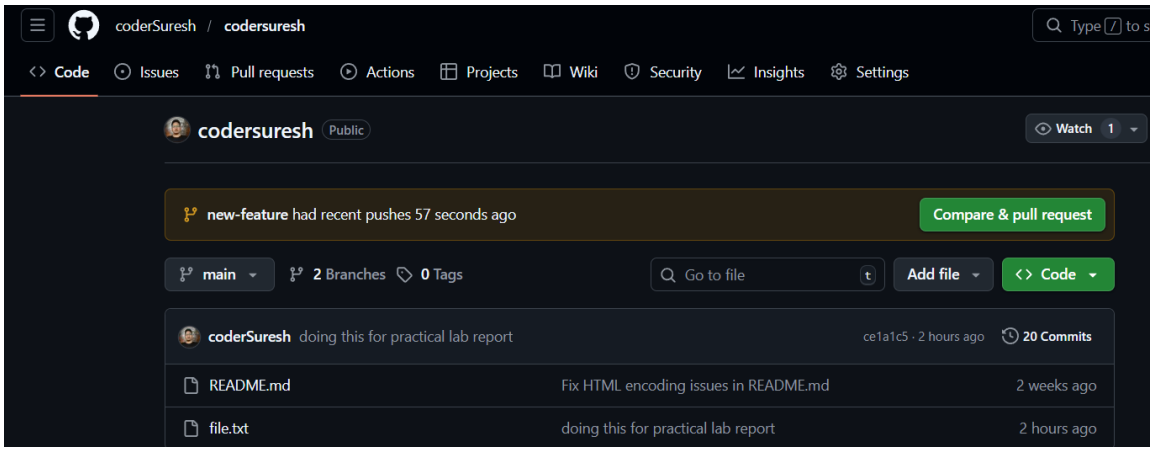
Collaborator Updates the Files:

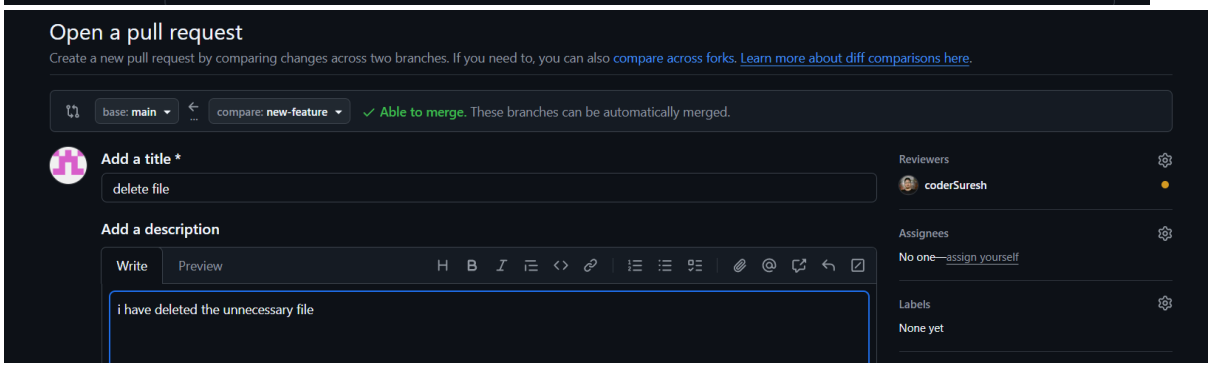
Collaborator adds, updates, or removes files and pushes the changes to new branch.

```
@thebbanepal ➜/workspaces/codersuresh (new-feature) $ git add .
@thebbanepal ➜/workspaces/codersuresh (new-feature) $ git commit -m "delete file"
[new-feature 082e368] delete file
 1 file changed, 1 deletion(-)
 delete mode 100644 file.txt
@thebbanepal ➜/workspaces/codersuresh (new-feature) $ git push origin new-feature
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Delta compression using up to 2 threads
Compressing objects: 100% (1/1), done.
```

Creating Pull Request:

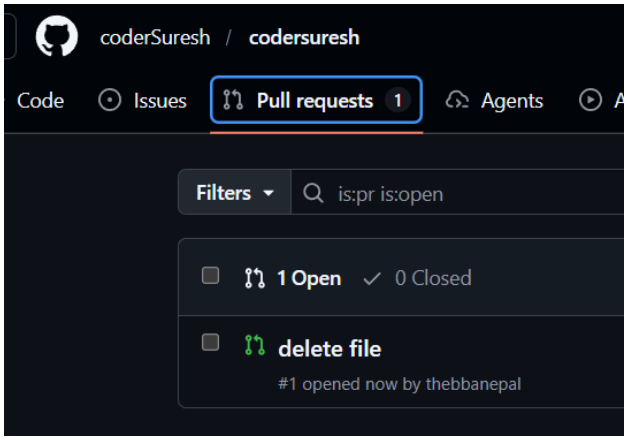
After pushing the changes, collaborator visits the GitHub repository and creates pull request by clicking on compare and pull request button. Reviewers can be assigned manually.



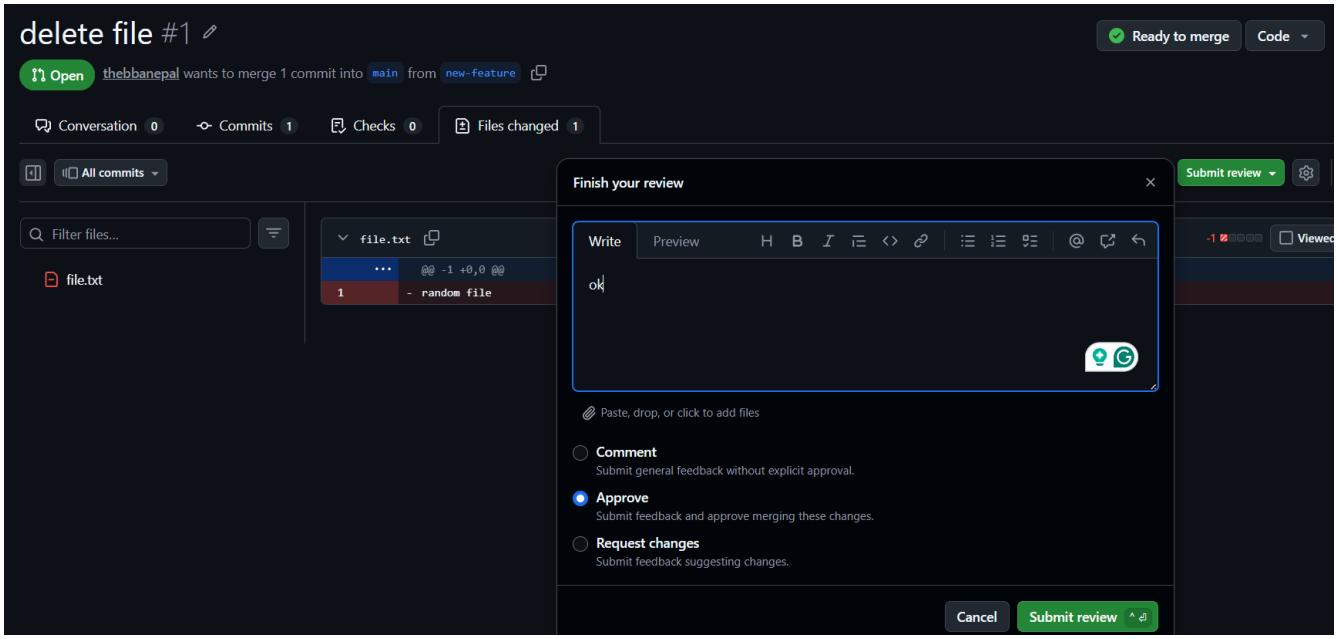


Reviewing Pull Request:

Owner of the repository can see pull requests.

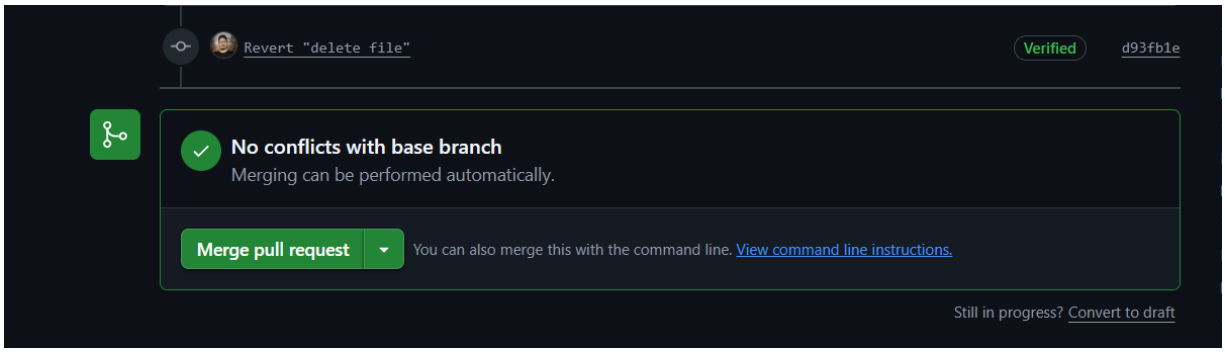


Owner reviews the pull request and merges the changes into main branch or request changes.



Merging with Main Branch:

Owner of the repository can click on the merge pull request button to merge changes from collaborator into the main branch.



There are three types of merges in GitHub:

1. Merge Commit: Combines all commits from the feature branch into the base branch by creating a new, dedicated merge commit to preserve full history.
2. Squash and Merge: Condenses all commits from the pull request into a single commit, keeping the main branch history clean and linear.
3. Rebase and Merge: Appends each individual commit onto the base branch without a merge commit, creating a flat, straight-line project history.

We use git merge command to merge the branch

```
@thebbanepal ➜/workspaces/codersuresh (new-feature) $ git checkout main
Switched to branch 'main'
Your branch is up to date with 'origin/main'.
@thebbanepal ➜/workspaces/codersuresh (main) $ git merge new-feature
Updating ce1a1c5..082e368
Fast-forward
 file.txt | 1 -
 1 file changed, 1 deletion(-)
 delete mode 100644 file.txt
```

Deleting the Branch:

We can delete a branch using git branch -d <branch_name> command.

```
@thebbanepal ➜/workspaces/codersuresh (main) $ git branch
* main
  new-feature
@thebbanepal ➜/workspaces/codersuresh (main) $ git branch -d new-feature
Deleted branch new-feature (was 082e368).
@thebbanepal ➜/workspaces/codersuresh (main) $ git branch
* main
@thebbanepal ➜/workspaces/codersuresh (main) $
```

RETRIEVE REMOTE DATA:

To bring local up to date, we use two commands git fetch and git pull.

Git Fetch:

It fetches from remote repository and updates only remote tracking branches such as origin/main without affecting the current local branch.

Git Pull:

It fetches from remote repository and merges with the current local branch. It can lead to merge conflict if local changes clash with remote changes. To prevent it, we can use git pull --rebase, which puts local changes on top of fetched changes.

```
@thebbanepal ➜/workspaces/codersuresh (main) $ git pull origin main
From https://github.com/coderSuresh/codersuresh
 * branch            main       -> FETCH_HEAD
Already up to date.
```

CONCLUSION:

Hence, by doing this lab, we learned about Git and GitHub. We learned how we can use these version control tools for repository management, branching, committing, team collaboration and working with remote repositories. This helps in efficient teamwork and proper management of software projects.